```
using Fruit;                                                    ❶

var client = new FruitClient(                                   ❷
    "https://localhost:7186",                                   ❸
    new HttpClient());                                          ❹

Fruit.Fruit created = await client.FruitPOSTAsync("123",        ❺
    new Fruit.Fruit { Name = "Banana", Stock = 100 });          ❻
Console.WriteLine($"Created {created.Name}");

Fruit.Fruit fetched = await client.FruitGETAsync("123");        ❼
Console.WriteLine($"Fetched {fetched.Name}");
```

❶ The code is generated in the Fruit namespace.

❷ Uses the generated FruitClient

❸ Specifies the base address of the API

❹ The provided HttpClient is used to call the API.

❺ Calls the MapPost endpoint of the API

❻ The Fruit type is generated automatically by NSwag.

❼ Calls the MapGet endpoint of the API

This code is simultaneously impressive and somewhat horrible:

- It's impressive that you're able to generate all the boilerplate code for interacting with the API. You don't have to do any string interpolation to calculate the path. You don't have to serialize the request body or deserialize the response. You don't have to check for error status codes. The generated code takes care of all those tasks.
- Those `FruitPOSTAsync` and `FruitGETAsync` methods have really ugly names!

Luckily, you can fix the ugly method names: improve your API's OpenAPI definition by adding `WithName()` to every API. The name you provide for your endpoint is used as the OperationID in the OpenAPI description; then NSwag uses it to generate the client methods. This scenario is a prime example of adding more metadata to your OpenAPI, making the tooling better for your consumers.